HAAP — Hawcx Agentic Authentication Platform

hawcx-manager

Build-Out Audit & Architecture Review

Multi-Call Super-Binary — BusyBox-Style Dispatch

Customer-Side Runtime & Employee-Laptop Enrollment

Audit date:	2026-05-28
SDK release line:	`v0.1.0-alpha.10`
Protocol spec:	HAAP CS v7.2.5
Crate home:	`hx_agent_client_auth_service`
Release channel:	`hawcx_agentic_sdk`
Multi-call cutover:	2026-05-22 (Phases 1–5)

Compiled from conversational audit dated 2026-05-28.

Spec references: `HAAP-Canonical-Specification-v7_2_5.md`, `DESIGN-MEMO-MULTICALL-BINARY.md`, `SDK-BUILD-WITH-HAWCX-MANAGER.md`.

Contents

List of Figures

List of Tables

1 Executive Summary

Verdict: production-ready. The `hawcx-manager` crate is structurally complete, spec-aligned (Phases 1–5 of `DESIGN-MEMO-MULTICALL-BINARY.md` landed 2026-05-22), and fully wired into the SDK release pipeline. All eight dispatch surfaces (`authenticator`, `tqs-precompute`, `tqs-jit`, `assembler`, `eib`, `supervisor`, `manager`, `sdk`) are implemented. Build is clean (zero warnings), all 65 tests pass.

One-line mental model

`hawcx-manager` is a BusyBox-style multi-call binary. It is one Rust ELF that, depending on `argv[0]` (the symlink it was invoked through) or `argv[1]` (its subcommand), behaves as one of eight roles. There is no runtime "manager process" that "talks to" the supervisor — *the supervisor is the same binary, invoked under a different name.*

Where the binary talks to the network

Three of the eight roles open a network socket; the other five are local-only.

Role	Endpoint	Transport	Purpose
`manager` (laptop)	CAA `/v1/enroll`	HTTPS + JSON	Employee enrollment
`supervisor` (container)	CAA supervisor-control	mTLS gRPC stream	Receive provisioning ops
`supervisor` (first launch)	AS `/v1/supervisor/bootstrap`	HTTPS + OTRC + CSR	Obtain own mTLS cert
`authenticator`, `tqs-*`, `assembler`, `eib`	—	UDS only	No outbound network

The naming nuance that causes confusion

The crate is named `hawcx-manager` but it is *not* a single "manager" program. The name *manager* historically denoted the employee-laptop enrollment CLI (`haap-manager`); in the 2026-05-22 cutover that program became one subcommand of a unified super-binary — and the super-binary inherited the name. The container-side supervisor was absorbed into the *same* ELF. Filenames such as `caa_channel.rs` in the supervisor crate further muddy the water: that channel targets the *Admin Orchestrator*, not the CAA REST endpoint that the laptop enrollment client talks to. §5 disentangles all three admin-side endpoints.

2 Crate Build-Out Status

2.1 Location and layout

The crate lives at hx_agent_client_auth_service/crates/hawcx-manager. It is a binary + library hybrid: `src/main.rs` is a 12-line entry point that calls into the library crate `hawcx_manager`.

```
hawcx-manager/
|-- Cargo.toml
|-- src/
|   |-- main.rs                          # 12 LOC entry: hawcx_manager::run(argv)
|   |-- lib.rs                           # 174 LOC top-level dispatch
|   |-- dispatch.rs                      # 218 LOC argv0/argv1 routing + 30 tests
|   |-- subcommands/
|   |   |-- mod.rs
|   |   |-- authenticator.rs             # thin -> haap_auth_bin::
    run_authenticator
|   |   |-- tqs_precompute.rs            # thin -> haap_tqs_precompute_bin::run
|   |   |-- tqs_jit.rs                   # thin -> haap_tqs_jit_bin::run
|   |   |-- assembler.rs                 # thin -> haap_assembler_bin::
    run_assembler
|   |   |-- eib.rs                       # thin -> haap_eib_bin::run_eib
|   |   |-- supervisor.rs                # thin -> haap_supervisor::run
|   |   |-- sdk/mod.rs                   # 266 LOC: full ported SDK CLI
|   |   `-- manager/                     # laptop-side enrollment (full port)
|   |       |-- mod.rs
|   |       |-- caa_client.rs            # HTTPS to CAA /v1/enroll
|   |       |-- config.rs
|   |       |-- device_flow.rs           # OIDC device-code flow
|   |       |-- ipc.rs
|   |       |-- keystore.rs              # OS keychain seal/unseal
|   |       |-- state.rs                 # ~/.hawcx/state.json
|   |       `-- commands/
|   |           |-- enroll.rs
|   |           |-- logout.rs
|   |           `-- status.rs
`-- tests/
    `-- smoke.rs                         # 3 integration tests (all pass)
```

2.2 Cargo.toml summary

One `[[bin]]` target (`hawcx-manager`) plus a library target (`hawcx_manager`). Direct dependencies on all six legacy `*-bin` library crates (`haap-auth-bin`, `haap-tqs-precompute-bin`, `haap-tqs-jit-bin`, `haap-assembler-bin`, `haap-eib-bin`) plus `haap-supervisor` (already a non-bin crate). Manager/SDK pull in `haap-sdk-types`, `haap-sdk-sealer`, `haap-redis`, plus the standard Tokio / Clap / Serde / Reqwest stack.

2.3 Per-surface implementation status

Surface	Wrapper	Status	Backing crate / module
`haap-authenticator`	thin	✓ complete	`haap_auth_bin::run_authenticator`
`haap-tqs-precompute`	thin	✓ complete	`haap_tqs_precompute_bin::run`
`haap-tqs-jit`	thin	✓ complete	`haap_tqs_jit_bin::run`
`haap-assembler`	thin	✓ complete	`haap_assembler_bin::run_assembler`
`haap-eib`	thin	✓ complete	`haap_eib_bin::run_eib`
`haap-supervisor`	adapter	✓ complete	`haap_supervisor::run` (returns `ExitCode`)
`haap-manager`	full port	✓ complete	`subcommands::manager` — 3 verbs: `enroll`, `status`, `logout`
`haap-sdk`	full port	✓ complete	`subcommands::sdk` — 5 verbs: `run-pipeline`, `run-rsv`, `substrate-fetch`, `seal`, `unseal`

Table 1: Per-role dispatch status. Zero `unimplemented!()`, `todo!()`, or `FIXME` markers anywhere in the crate.

2.4 Build, tests, and toolchain

- `cargo check -p hawcx-manager` — passes in ≈7.8 s, zero warnings.
- `cargo build -release -p hawcx-manager` — succeeds, single ≈40 MB binary (versus the prior ≈90–180 MB spread across nine separate binaries).
- Unit tests: 62 pass — 30 dispatch routing tests + 32 manager / SDK domain tests (config, keystore, IPC, device-flow, state roundtrips).
- Integration: `tests/smoke.rs` — 3 pass (`-version`, `-help` lists all 13 subcommands, unknown-subcommand exits nonzero).

2.5 Spec alignment

Cross-checked against three canonical-spec documents:

- `HAAP-TOPOLOGY-MAPPING.md` — confirms `hx_agent_client_auth_service` owns the multi-call super-binary.
- `DESIGN-MEMO-MULTICALL-BINARY.md` — Phases 1–5 landed 2026-05-22. Phase 6 (deleting deprecated `[[bin]]` targets from the six `*-bin` shims) is intentionally deferred.
- `SDK-BUILD-WITH-HAWCX-MANAGER.md` — Dockerfile + release workflow describe the symlink-fan-out post-cutover.

2.6 Release-channel integration

- `Dockerfile` line 64 builds only the `hawcx-manager` binary target; lines 71–79 create seven symlinks (`haap-authenticator`, `haap-tqs-precompute`, `haap-tqs-jit`, `haap-assembler`, `haap-eib`, `haap-supervisor`, `haap-sdk`); ENTRYPOINT defaults to `haap-supervisor`.
- `.github/workflows/release.yml` `build-distribution` job builds the single target, packages with symlinks (Unix) or `.exe` copies (Windows), uploads as `docker-bins`; the Docker job re-creates symlinks after download.

3 Architecture: Multi-Call Binary Dispatch

The crate implements the classic BusyBox pattern: a single executable inspects `argv[0]` at startup, looks up a role by basename, and branches into the role-specific entry point. Eight legacy command-line names plus the canonical `hawcx-manager` name all resolve into the same `.text` segment.

3.1 Dispatch logic

```rust
// crates/hawcx-manager/src/dispatch.rs
pub enum Role {
    Authenticator, TqsPrecompute, TqsJit,
    Assembler, Eib, Supervisor,
    Manager, Sdk,
    Direct,                          // unknown argv[0] -> route via argv[1]
}

pub fn role_from_argv0(argv0: Option<&str>) -> Role {
    let stem = basename(argv0).strip_suffix(".exe").unwrap_or(basename);
    match stem {
        "haap-authenticator"  => Role::Authenticator,
        "haap-tqs-precompute" => Role::TqsPrecompute,
        "haap-tqs-jit"        => Role::TqsJit,
        "haap-assembler"      => Role::Assembler,
        "haap-eib"            => Role::Eib,
        "haap-supervisor"     => Role::Supervisor,
        "haap-manager"        => Role::Manager,
        "haap-sdk"            => Role::Sdk,
        _                     => Role::Direct,
    }
}
```

When `argv[0]` is the canonical name (`hawcx-manager`) the dispatcher falls through to `argv[1]` subcommand routing (`enroll`, `status`, `authenticator`, `run-pipeline`, ...). When `argv[0]` is a legacy symlink, the role is dispatched directly with the full argv preserved so the underlying daemon sees its conventional argv shape for logging and clap parsing.

3.2 Figure: dispatch fan-out from a single ELF

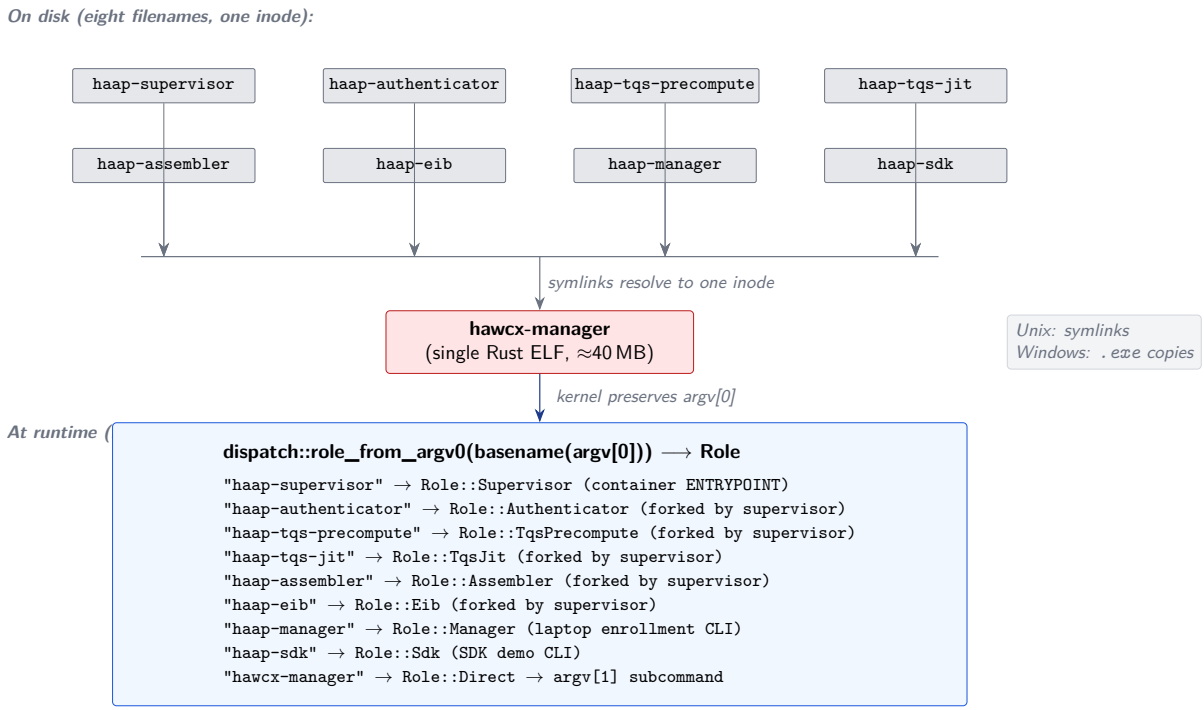

Figure 1: BusyBox-style multi-call fan-out. Eight symlinks (or `.exe` copies on Windows) all resolve to the same `hawcx-manager` ELF on disk. At runtime, the kernel preserves `argv[0]` across `exec()`; the dispatcher branches on `basename(argv[0])` into one of nine roles (eight legacy names plus the canonical `hawcx-manager` which routes by `argv[1]`).

4 Two Deployment Contexts, One Binary

The same `hawcx-manager` ELF is deployed in two physically distinct contexts, with completely different lifecycles:

- **Customer container** (`ghcr.io/hawcx/hx-agent-sdk`) — long-running. ENTRYPOINT is the `haap-supervisor` symlink, which dispatches to `Role::Supervisor`; that role then forks five children, each of which is also `hawcx-manager` invoked under a different legacy symlink name.
- **Employee laptop** — one-shot. Operator runs `hawcx-manager enroll ...` (or the legacy `haap-manager` symlink). Talks to the CAA REST endpoint over HTTPS, seals the returned credentials into the OS keychain, exits.

4.1 Figure: customer-container fork tree

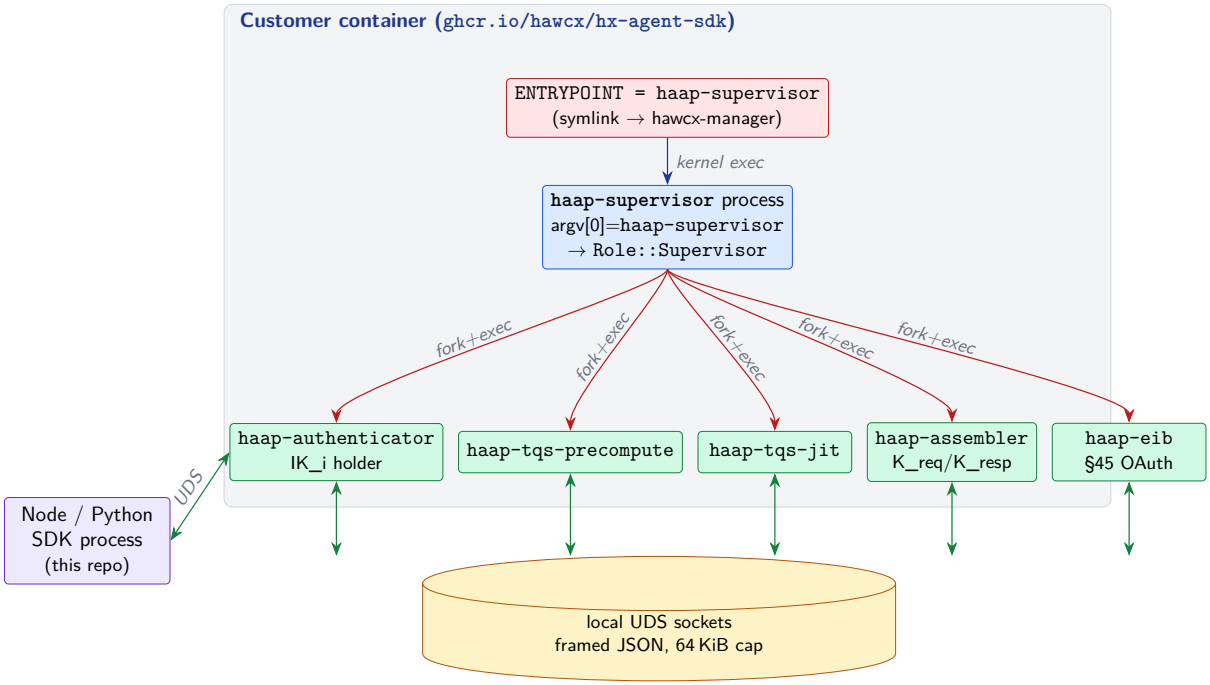

Figure 2: Container-side process tree. The supervisor is the only entry point; it forks the five daemons, each of which is the *same* `hawcx-manager` ELF invoked under a different legacy symlink name. All inter-daemon traffic is local UDS.

4.2 Figure: employee-laptop enrollment flow

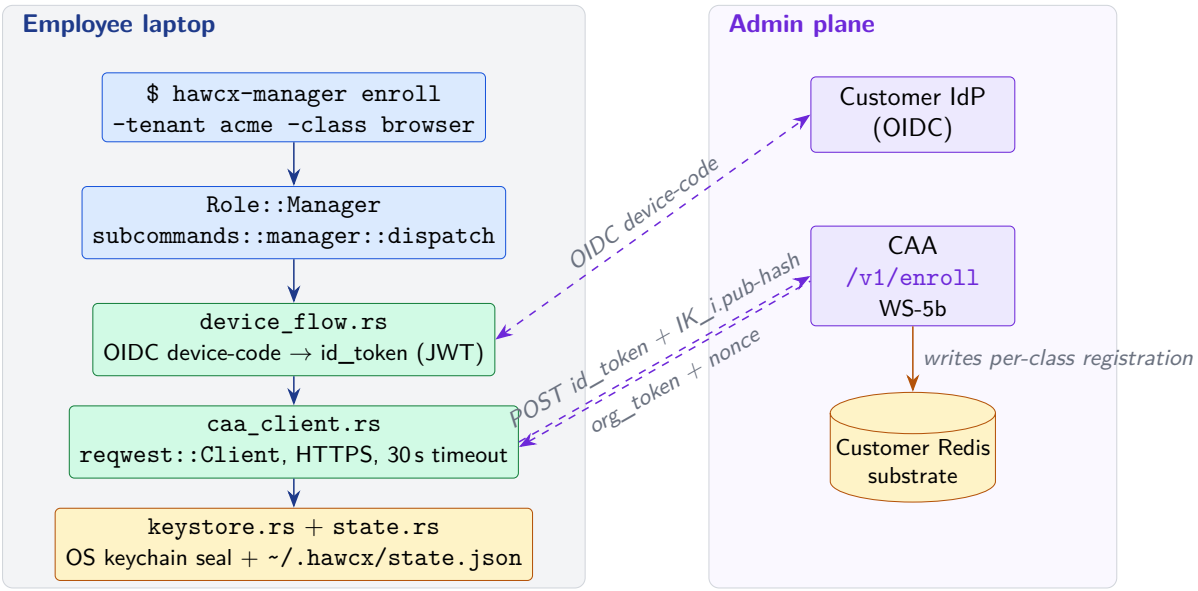

Figure 3: Employee-laptop enrollment. The `manager` role talks to the customer IdP (OIDC device-code) and then to the CAA's `/v1/enroll`. The CAA, not the laptop, writes the resulting registration into the customer Redis substrate; the supervisor inside the container picks it up from Redis on a later startup.

5 Admin-Plane Communication

5.1 The customer-side network boundary: CAA is the single edge

The customer-side runtime talks to **one cloud-edge service**: the **CAA** — the Client Admin Authenticator, sourced from `hx_agent_client_admin_service`. The CAA exposes *two distinct surfaces* consumed by the multi-call binary — a REST endpoint for laptop enrollment and a gRPC supervisor-control endpoint for the container supervisor. Anything further inside the admin plane — the *Admin Orchestrator*, the Auth Server, audit storage — sits **behind** the CAA and is reached only via CAA-internal hops. The customer-side runtime never opens a socket to those backend services directly.

> The env var that configures the supervisor's gRPC endpoint is `HAAP_CAA_ORCHESTRATOR_URL` — read it as "the CAA's orchestrator-facing endpoint," not "the Admin Orchestrator URL." The CAA is what answers that connection; it proxies operations to and from the Admin Orchestrator internally on the admin plane. The supervisor crate's filename `caa_channel.rs` reflects this accurately — the peer on the wire is the CAA.

The one exception is the supervisor's first-launch OTRC bootstrap, which talks to the Auth Server's `/v1/supervisor/bootstrap` endpoint directly. The CAA does not front this call because at first launch the supervisor does not yet possess the mTLS material the CAA gRPC surface requires.

Endpoint	Transport	Caller & purpose
CAA `/v1/enroll` WS-5b enrollment endpoint	HTTPS + JSON	`Role::Manager` on employee laptop. Enroll an agent class; receive signed org_token + registration nonce. Two auth modes: OIDC device-flow (`id_token` in body) or MDM (mTLS client cert). Endpoint hosted by `hx_agent_client_admin_service`.
CAA supervisor-control (gRPC) CAA's orchestrator-facing surface	mTLS gRPC streaming	`Role::Supervisor` in container. Persistent outbound channel *to the CAA* (configured by `HAAP_CAA_ORCHESTRATOR_URL`). Sends `RegisterSupervisor`, opens `OperationStream`, receives `SupervisorOperation` provisioning bundles, returns `DeliveryResult`/`RevokeResult`, reports `SupervisorStatus` ticks. The CAA proxies these to the Admin Orchestrator internally.
AS supervisor bootstrap §4.6.3 – first-launch only	HTTPS + OTRC + CSR	`Role::Supervisor` on first launch only. Generates ECDSA P-256 keypair locally, POSTs `{otrc, supervisor_csr_pem, label, org_id}` to AS `/v1/supervisor/bootstrap` *directly*. Persists the returned signed cert, zeroizes the OTRC env var. Fail-closed on error.

Table 2: Customer-side network surface. The CAA is the only cloud edge the supervisor talks to in steady state, with the Admin Orchestrator sitting behind it inside the admin plane. The AS bootstrap endpoint is a one-shot exception used only when the supervisor has no mTLS material yet.

5.2 Figure: full customer-side network topology

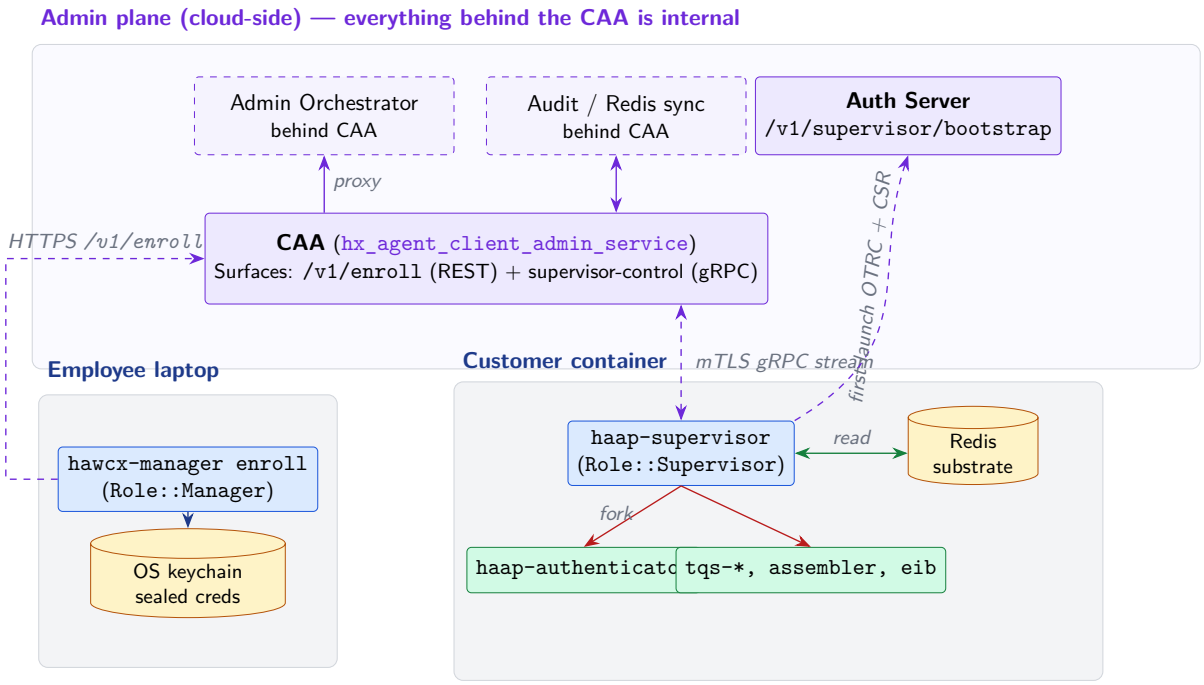

Figure 4: End-to-end customer-side network topology — **corrected**. The CAA is the single cloud edge the customer-side runtime talks to in steady state. Both the laptop enrollment client and the container supervisor open sockets *to the CAA*; the Admin Orchestrator and other admin-plane backends sit *behind* the CAA and are reached only via CAA-internal proxying. The only direct call past the CAA is the supervisor's first-launch OTRC bootstrap to the Auth Server, which exists precisely because the supervisor has no mTLS material yet for the CAA gRPC surface. Solid green arrows are local UDS; dashed purple arrows are network traffic.

5.3 Supervisor mTLS bootstrap states

The supervisor classifies its startup environment into one of four decisions (`caa_bootstrap.rs:88`):

- **`Disabled`** — `HAAP_CAA_ORCHESTRATOR_URL` unset. Legacy / standalone deployment; no admin channel.
- **`Enabled`** — URL set, cert + key files present. Normal happy path: open the gRPC channel and proceed.
- **`Degraded`** — URL set, material missing, but `HAAP_CAA_ALLOW_DEGRADED=true` is set. Explicit operator opt-in to legacy mode despite the URL being configured.
- **`BootstrappingViaOtrc`** (WS-3) — URL set, material missing, but `HAAP_SUPERVISOR_BOOTSTRAP_OTRC` and `HAAP_AS_BOOTSTRAP_URL` are present. Generates ECDSA P-256 keypair locally, builds CSR, POSTs to the AS bootstrap endpoint, persists the returned cert, *zeroizes the OTRC env var so a restart cannot replay the consumed code*. **Fail-closed** on any error — a half-bootstrapped supervisor sitting in degraded mode while an attacker re-uses the leaked OTRC would be worse than refusing to start.

5.4 Figure: supervisor bootstrap decision tree

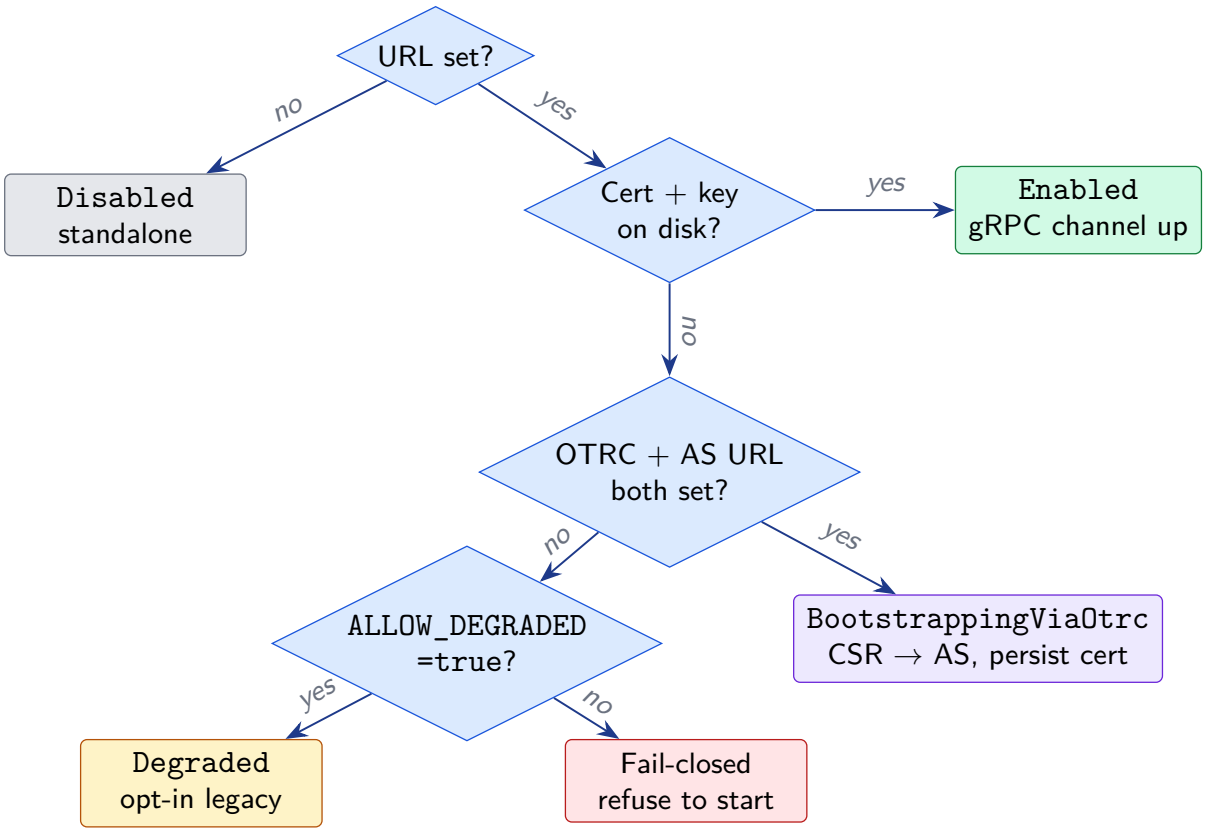

Figure 5: Supervisor mTLS bootstrap classifier. Four terminal states; the WS-3 OTRC path is the only mode that performs a CSR exchange with the Auth Server.

6 Registration Flow: §4.6.3

The §4.6.3 registration flow is where the user's earlier question pointed: *how does the authenticator's IK_i public key end up referenced in the signed org_token, and what role does the supervisor play?*

6.1 Key invariant

The supervisor is a *pass-through with capacity / lifecycle accounting* — not a cryptographic participant. `caa_relay.rs` states this explicitly: *"The supervisor is a pass-through; it does not inspect or modify the cryptographic content."* IK_i is generated inside the authenticator during the prepare-handshake; the supervisor only routes the provisioning bundle (received *from the CAA*, originally minted by the Admin Orchestrator that sits behind the CAA) down to the authenticator over UDS.

6.2 Who does what

- **Authenticator** (during prepare-handshake) — mints IK_i + IK_i.pub locally, persists it under a slot keyed by `prepared_agent_id`.
- **Upstream** (separate path, prepare-handshake) — the IK_i.pub hash is reported upstream during the prepare phase. *Not* via the supervisor's gRPC relay.
- **Admin Orchestrator** (behind the CAA) — signs the org_token, referencing the bound IK_i.pub hash; hands the bundle to the CAA, which streams it to the supervisor over the persistent mTLS gRPC channel. The supervisor never sees the Orchestrator on the wire directly.
- **CAA** — the cloud edge. Terminates the supervisor's mTLS gRPC connection, proxies operations to and from the Admin Orchestrator on the admin plane, surfaces `SupervisorOperation` payloads downward to the supervisor.
- **Supervisor** — receives the `SupervisorOperation` bundle from the CAA, looks up the agent's control socket via the canonical path convention (`paths::ipc_path(..., "auth-control")`), sends `MSG_REGISTER_REQ` (0x07) over UDS carrying `{agent_class, subject_user_id, prepared_agent_id}`. Reports `DeliveryResult` back to the CAA. Updates `AgentTracker` for the next `SupervisorStatus` tick.
- **Authenticator** (on `MSG_REGISTER_REQ`) — the `prepared_agent_id` field tells `perform_agent_registration` to *reuse* the IK_i already persisted under that slot, pair it with the inbound signed org_token, and complete the registration against the Auth Server. v7.1.1-aware authenticators fall through to the legacy mint-fresh path if no IK_i is persisted; pre-v7.1.1 authenticators ignore the extra field.

6.3 Figure: sequence diagram — agent registration

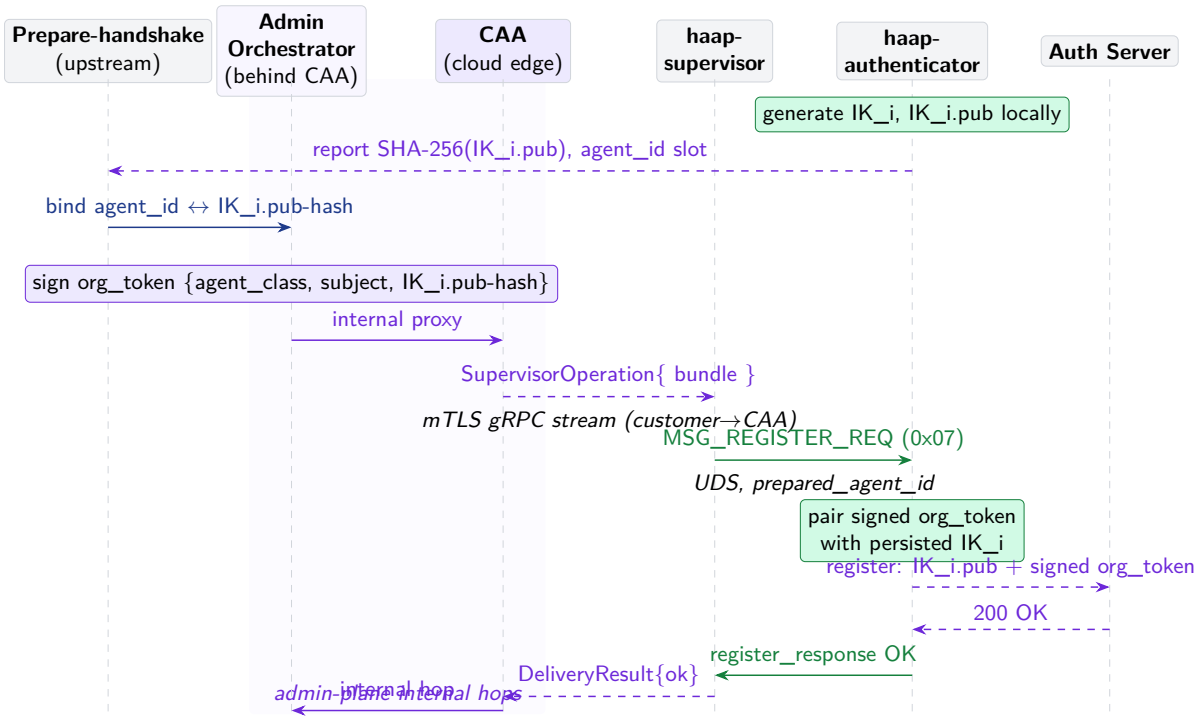

Figure 6: Sequence diagram for §4.6.3 agent registration — **corrected**. The supervisor's wire peer is the **CAA**, not the Admin Orchestrator directly. The Orchestrator signs the org_token inside the admin plane; the CAA proxies the provisioning bundle outward to the supervisor over mTLS gRPC, and proxies the supervisor's `DeliveryResult` back inward. Solid green arrows are local UDS; dashed purple arrows cross the customer↔CAA wire; solid purple arrows are admin-plane internal hops the customer-side runtime never sees.

7 What Was Wrong in the First Two Passes

First-pass error. The initial audit response asserted that *"the container-side daemons never reach out to the CAA."* That was wrong. The `haap-supervisor` role maintains a persistent outbound *mTLS gRPC stream to the CAA*, and (under the WS-3 first-launch path) also POSTs a CSR directly to the Auth Server. Four supervisor files (`caa_bootstrap.rs`, `caa_channel.rs`, `caa_relay.rs`, `otrc_bootstrap_client.rs` — 1,726 LOC) implement this and were missed in the original sweep.

Second-pass error. The corrected first response then described the supervisor's gRPC peer as the *Admin Orchestrator* (and proposed renaming the `caa_*` files to `orchestrator_*`). That was also wrong, in the opposite direction. The supervisor's wire peer is the **CAA**, which proxies operations to/from the Admin Orchestrator *internally on the admin plane*. The customer-side runtime never opens a socket to the Orchestrator directly. The env var name `HAAP_CAA_ORCHESTRATOR_URL` encodes this precisely: the CAA's orchestrator-facing endpoint, not the Orchestrator's own endpoint. The filename `caa_channel.rs` is accurate as written; the proposed rename would have been a regression.

7.1 Source of the errors

- **First-pass:** the initial grep covered the `manager` subtree only, scoped by the laptop-enrollment framing of the question. Skipping `haap-supervisor/` omitted the entire 1,700-LOC outbound-network block.
- **Second-pass:** after finding `caa_channel.rs` I latched onto the in-file comment *"supervisor connects outbound to Admin Orchestrator"* as authoritative. That in-source phrasing is a shorthand for the upstream destination of the operation, not the wire peer; reading just the channel comment without tracing the actual TLS hop and env var produced the wrong topology. The CAA terminates the connection; the Admin Orchestrator is the logical origin of the operations the CAA forwards.
- **Methodology fix:** when reasoning about cloud-edge topology, anchor on the env var that supplies the URL and the TLS peer cert, not on the in-source narrative comment. Both artifacts here named the CAA correctly; the prose summary misled.

8 Bottom Line

`hawcx-manager` is production-ready as a multi-call super-binary. All eight dispatch surfaces are wired, builds cleanly, 65/65 tests pass, fully integrated into the SDK Docker image and release workflow. Spec Phases 1–5 of the multi-call cutover landed 2026-05-22; Phase 6 (deleting deprecated `[[bin]]` targets from the six legacy `*-bin` shims) is intentionally deferred.

8.1 Network surface, restated

Three of the eight roles touch the network — and the customer-side runtime touches only **two cloud-edge services**: the CAA in steady state, and (one-shot) the Auth Server for first-launch bootstrap:

1. `Role::Manager` (laptop) → **CAA** `/v1/enroll` — one-shot HTTPS per enrollment.

2. `Role::Supervisor` (container) → **CAA** supervisor-control — persistent mTLS gRPC stream, `RegisterSupervisor` + `OperationStream` + `ReportSupervisorStatus`. CAA proxies these to the Admin Orchestrator behind it on the admin plane.
3. `Role::Supervisor` (container, first launch only) → **AS** `/v1/supervisor/bootstrap` — OTRC + CSR exchange to provision the supervisor's own mTLS material. Direct to AS because the supervisor has no mTLS cert yet for the CAA gRPC surface. Fail-closed on error.

8.2 Top items to monitor post-launch

- **Phase 6 cleanup:** remove the `[[bin]]` targets from the six deprecated `*-bin` shim crates once external invocations are confirmed migrated to the multi-call binary.
- **Per-role memory footprint:** a single fat binary means each forked role loads all dependencies in its address space; fine on current targets but worth a baseline RSS measurement at GA.
- **Audit logging coverage:** confirm that the supervisor's `DeliveryResult` reports flowing through the CAA contain enough detail for the Admin Orchestrator to reconstruct §4.6.3 audit trails without round-tripping the customer Redis substrate.
- **Documentation clarity (not code):** the in-source comments in `caa_channel.rs` should be tightened to explicitly state that the wire peer is the CAA and that the Admin Orchestrator is the upstream logical origin (only) of the operations the CAA proxies. The filenames themselves are correct.

End of report — hawcx-manager Build-Out Audit, 2026-05-28.
Compiled from in-session investigation against `hx_agent_client_auth_service/crates/hawcx-manager` and `hx_agent_client_auth_service/crates/haap-supervisor`.